

Отчёт. Cut Cross-Entropy на форме Gemma 2 2B

Статья: Erik Wijmans, Brody Huval, Alexander Hertzberg, Vladlen Koltun, Philipp Krähenbühl. Cut Your Losses in Large-Vocabulary Language Models. ICLR 2025. arXiv:2411.09009, <https://arxiv.org/abs/2411.09009>. Код - сабмодуль ml-cross-entropy, коммит 3de376c.

Постановка

Форма Gemma 2 2B из статьи: скрытый размер $D = 2304$, словарь $|V| = 256000$, softcap 30, тип bf16. Число токенов в одном вызове: 128, 256, 512, 1024, 2048, 4096, 8192. Точка статьи - 8192.

Три метода:

- CCE - `linear_cross_entropy` с `impl="cce"`;
- `torch_compile` - тот же вызов с `impl="torch_compile"`, контроль из статьи;
- baseline - `logits = E @ C.T`, затем `F.cross_entropy` по fp32-логитам.

Е и С случайные. Размер матрицы логитов от значений не зависит, для памяти этого достаточно. Время backward у CCE на случайных тензорах нельзя сравнивать напрямую с числами из оригинальной статьи: softmax плоский, фильтр градиента выкидывает почти все блоки. Ниже по времени приведён forward+backward как факт на этой карте, а не как повтор миллисекунд A100.

Карта - RTX 5080, 16 ГиБ. PyTorch 2.8.0, Triton 3.4.0. В статье снята на A100 80 ГиБ, PyTorch 2.4.1, CUDA 12.4.

Пунктир на графике памяти - одна копия логитов в fp32: $N \times 256000 \times 4$ байта. На 8192 токенах это 8000 МиБ.

Память forward + backward

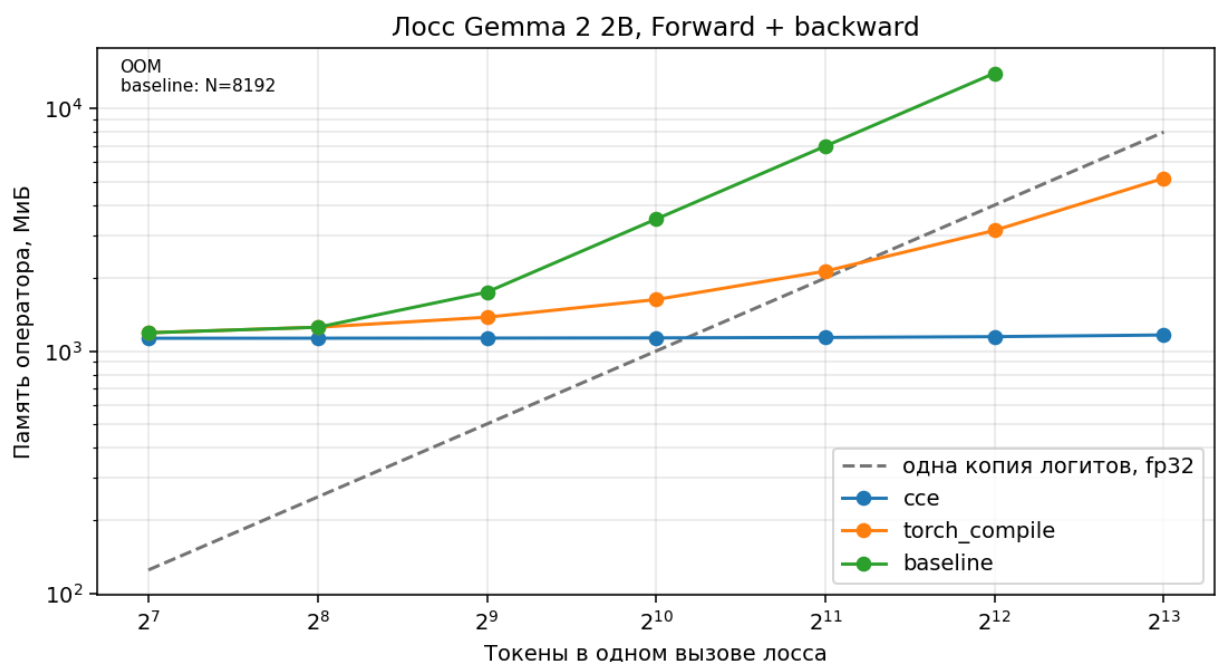


Рисунок 1. Память оператора, forward и backward.

Таблица 1. Пик памяти оператора, МиБ

Токены	CCE	torch.compile	baseline	одна копия fp32
128	1129	1189	1189	125
256	1129	1253	1253	250
512	1131	1378	1750	500
1024	1133	1631	3500	1000
2048	1137	2135	7000	2000
4096	1146	3144	14000	4000
8192	1164	5162	OOM	8000

Наблюдения:

- CCE почти не растёт: 1129 МиБ при 128 токенах и 1164 МиБ при 8192. Прирост 35 МиБ - это градиент по E. Градиент классификатора один и тот же у всех методов, $256000 \times 2304 \times 2$ байта ≈ 1125 МиБ, и он обязателен. CCE добавляет к нему единицы мегабайт. В оригинальной статье нижняя граница forward+backward - 1161 МБ, у CCE 1164 МБ. Здесь на 8192 токенах те же 1164.3 МиБ.
- baseline растёт вместе с $N \times |V|$. На 4096 токенах пик 14000 МиБ - это три с половиной fp32-копии, и карта уже почти полная. На 8192 токенах вызов не помещается. В статье на этом размере baseline занимает 28 ГБ, forward отдельно - 24 ГБ. На 16 ГиБ это и есть OOM. Линейное продолжение нашего forward: на 4096 токенах baseline занимает 12000 МиБ, на 8192 было бы 24000 МиБ, цифра из оригинальной статьи.
- torch.compile тоже растёт, но медленнее baseline. На 8192 токенах forward+backward - 5162 МиБ против 1164 у CCE, то есть в 4.4 раза больше. Forward по отдельности совпал со статьёй: 4000.1 МиБ против 4000 МБ в оригинальной статье. Combined-пик в статье 16 ГБ, у нас 5.2 ГБ: PyTorch 2.8 не держит одновременно столько копий, сколько 2.4.1. Качественный порядок тот же. CCE остаётся у нижней границы, baseline на точке статьи не запускается.

Время forward + backward

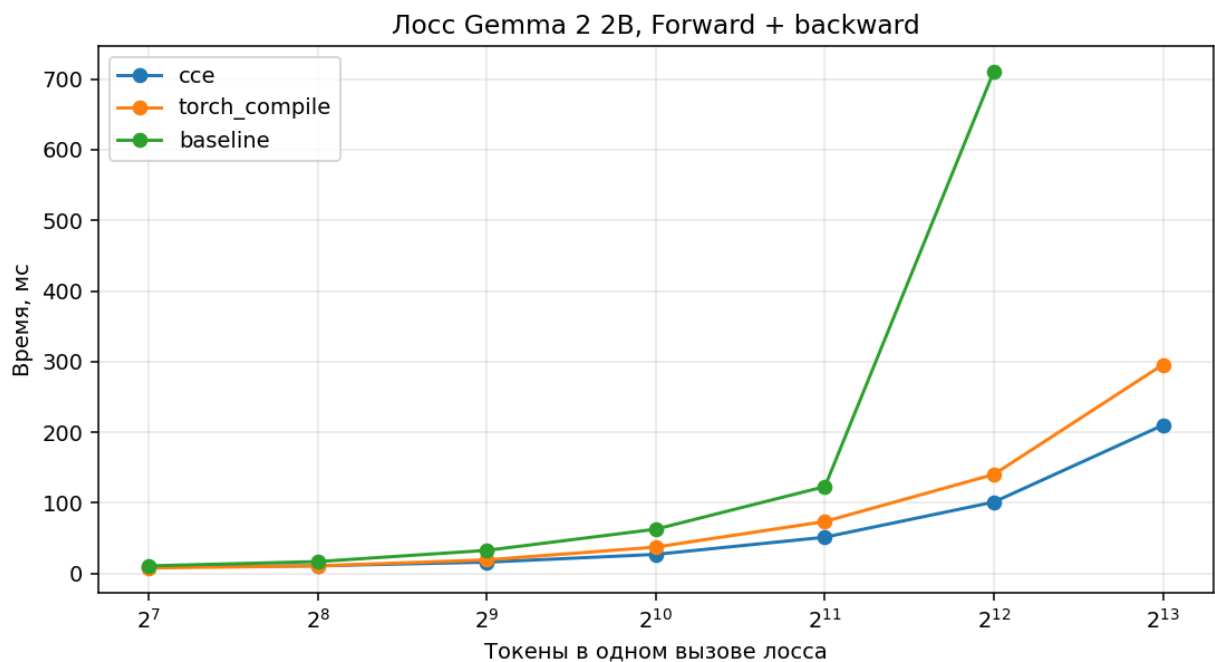


Рисунок 2. Время оператора, forward и backward.

Таблица 2. Время оператора, мс

Токены	CCE	torch.compile	baseline
128	9.8	8.0	10.9
512	16.2	19.4	32.9
1024	27.2	37.5	62.9
2048	51.5	73.7	123
4096	101	140	711
8192	210	296	OOM

Наблюдения:

- До 2048 токенов все три метода растут примерно линейно, baseline примерно вдвое медленнее CCE. На 4096 токенах baseline прыгает до 711 мс: 14 ГБ оператора на 16 ГБ карте, вызов упирается в память. CCE на 8192 токенах - 210 мс, torch.compile - 296 мс. Экономия памяти здесь не оплачена замедлением.
- Абсолютные миллисекунды со статьёй не совпадают. Там A100 и обученные активации Gemma: CCE и torch.compile оба около 145 мс на 8192 токенах. У нас другая карта и случайный softmax. Утверждение статьи, которое эти кривые проверяют, - CCE не медленнее сплавленного контроля и сильно легче наивной матрицы логитов. На этом прогоне оно выполняется.

Вывод:

Память наивного лосса - это $N \times |V|$, несколько копий. У Gemma 2 2B словарь 256000, поэтому уже несколько тысяч токенов в одном вызове стоят гигабайты, а 8192 токена на 16 ГБ не считаются. CCE считает логиты блоками в SRAM и в глобальную память пишет скаляр на токен плюс градиенты E и C. Поэтому его кривая плоская и на 8192 токенах совпадает с нижней границей из оригинальной статьи.